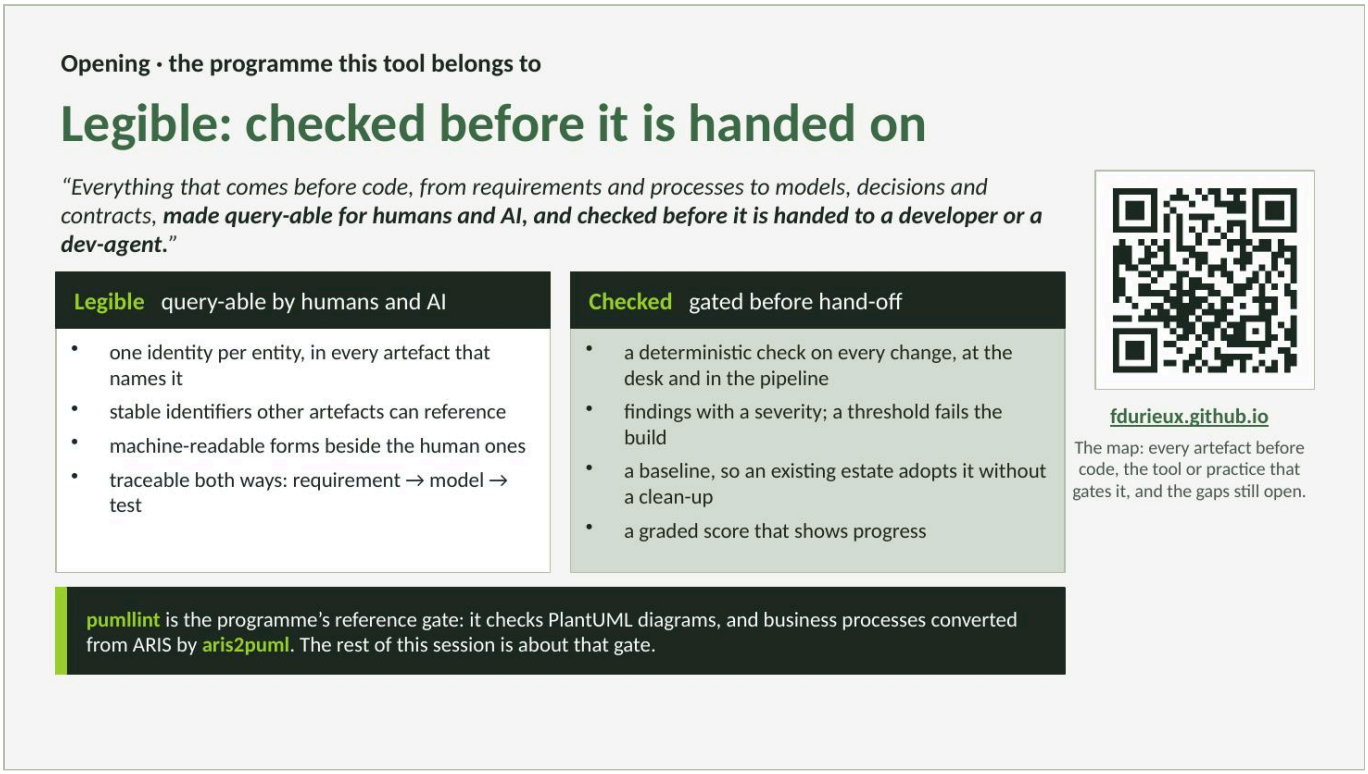
Legible for analysts · semantic linting of PlantUML diagrams

35 min presenting, 9 min room discussion, 1 min buffer (0:44–0:45). If time runs short, take slides 9–11 at a minute each; keep the live demo (slide 15) whole, and the discussion on slides 3 and 8.

Part	Clock	#	Slide	Min	Of which discussion	Room
Opening 5 min	0:00–0:03	1	Legible: checked before it is handed on	3		
	0:03–0:05	2	Three questions for this round table	2	1	Warm-up: who writes, who builds on diagrams?
① The problem 13 min	0:05–0:09	3	Problems to solve, across the delivery pipeline	4	1	Which of these do you see in your own work?
	0:09–0:11	4	What we mean by semantic correctness	2		
	0:11–0:14	5	Both render. Which branch approves?	3	1	Which branch approves?
	0:14–0:16	6	Both render. Which one is right?	2	1	Which one is right?
	0:16–0:18	7	Semantic quality: why it matters now	2		
② The contribution 10 min	0:18–0:22	8	Tooling across the delivery pipeline	4	1	Which gates already run in your pipeline?
	0:22–0:24	9	From linter to static model analysis	2		
	0:24–0:25	10	Code smells, model smells	1		
	0:25–0:27	11	Detect, measure, govern	2		
	0:27–0:28	12	Does the level matter? Measured: yes	1		
③ In practice 14 min	0:28–0:29	13	The demo: a credit application	1		
	0:29–0:31	14	Valid or not, and why	2		
	0:31–0:40	15	Live demo: localhost:8000	9	3	Who would have found these, and when?
	0:40–0:42	16	Diagram quality, built into the workflow	2		PoC: what must it show? Where's the gate?
Closing 2 min	0:42–0:44	17	A PoC on semantic linting: go or no-go?	2	1	Vote 👍 / 👎; if go: scope, criteria, owner, check-in
	0:44–0:44	18	pumllint on GitHub: where to read more	0		
Buffer	0:44–0:45		Overrun, questions, close	1		

Legible: checked before it is handed on



SAY

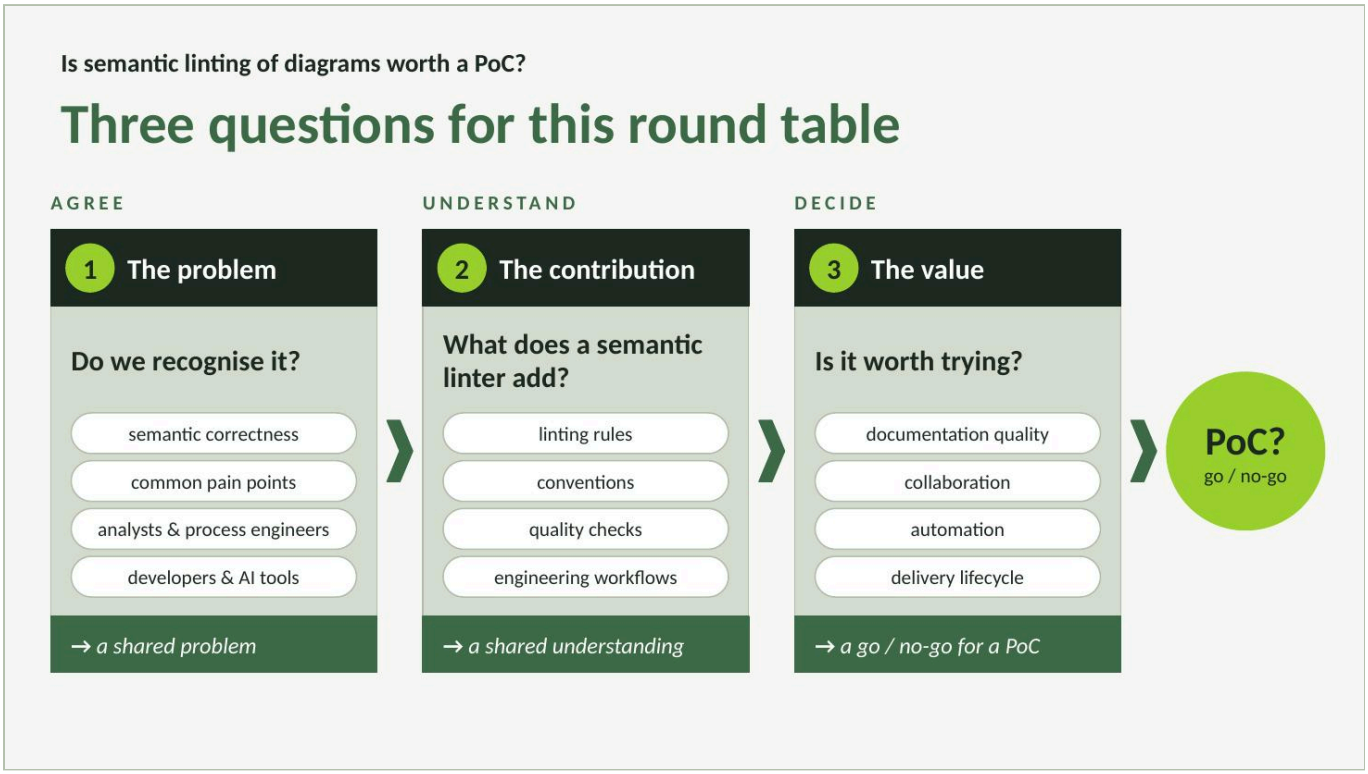
- This session sits in a wider programme, Legible: everything written before code (requirements, processes, models, decisions, contracts) made query-able for humans and AI, and checked before anyone builds on it.
- Two halves. Legible: one identity per entity, stable IDs, machine-readable, traceable both ways. Checked: a gate before hand-off, with a severity, a threshold, a baseline and a score.
- Code has had those gates for twenty years. The information code is built from has had none, and AI now builds code from it at near-zero cost.
- pumllint is the programme's reference gate. Today is about that gate, from an analyst's point of view.

TIP

The QR code opens the programme's map; leave it up a few seconds for phones.

Next → Three questions frame the next forty minutes.

Three questions for this round table



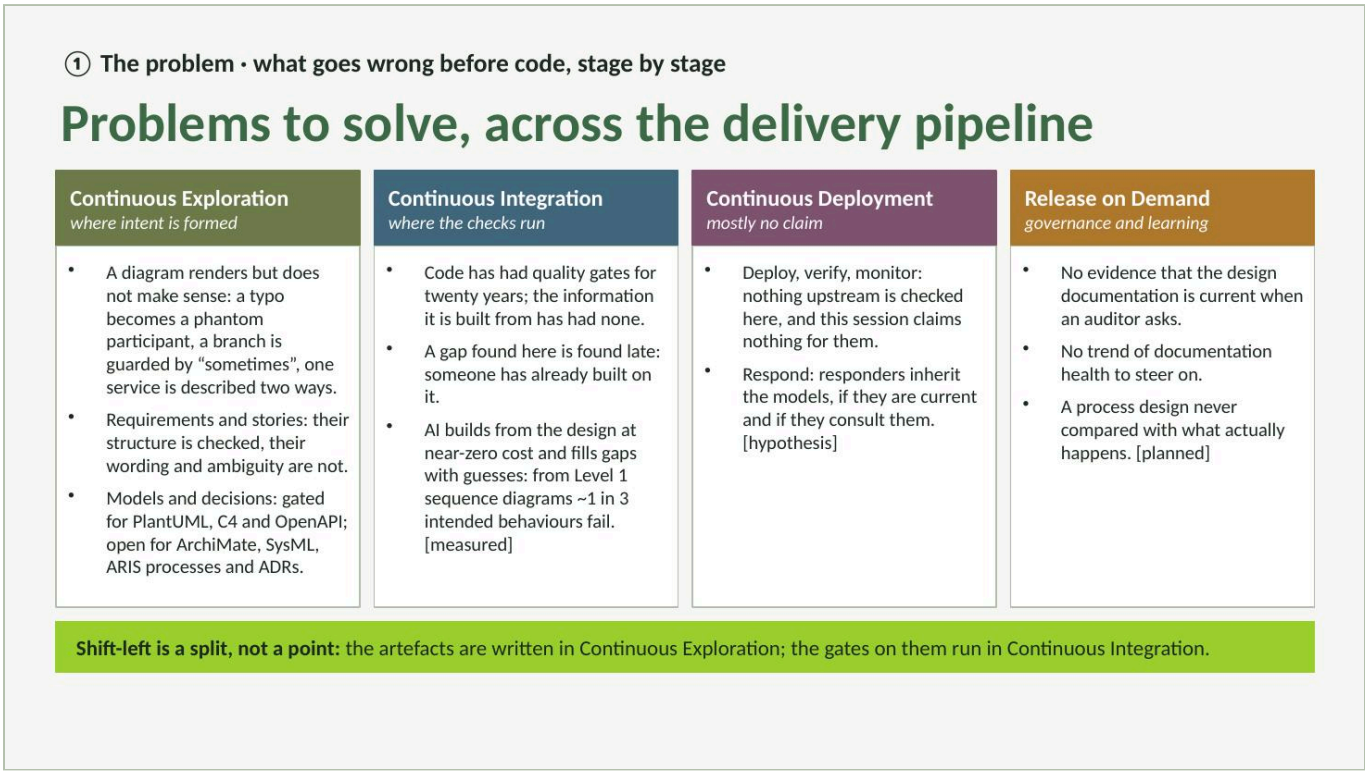
SAY

- Three questions, three outcomes: agree that there is a problem, understand what a semantic linter adds, decide whether a PoC is worth it.
- The chips are the keywords from the programme, sorted by the question they belong to.
- We end at a go / no-go for a PoC, not at adoption.

> **ASK**
>
> Warm-up, show of hands: who writes PlantUML diagrams? Who builds on someone else's diagram: a process, a design, code?

Next → First: what actually goes wrong before code?

Problems to solve, across the delivery pipeline



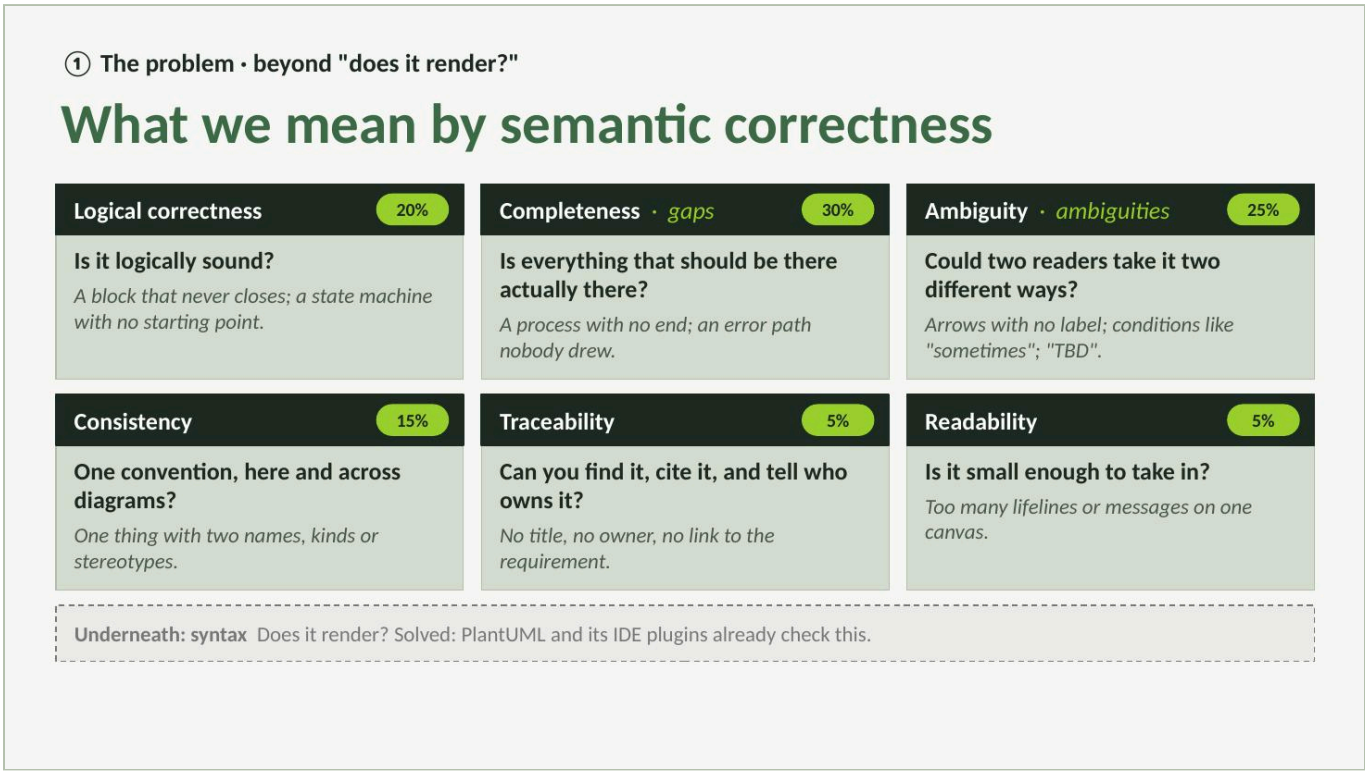
SAY

- The four aspects of the delivery pipeline, as SAFe names them; used as vocabulary only.
- Continuous Exploration is where analysts work: requirements, processes, models. Every problem in that column is something that renders, reads well, and still leaves the next person guessing.
- Continuous Integration is where checks run today: on code, for twenty years. The design the code comes from is not checked, so its gaps are found late, or filled by an AI guess. The one measured number: from Level 1 sequence diagrams, about 1 in 3 intended behaviours fail.
- Deployment carries no claim; Release on Demand is about evidence and learning.
- The bottom line is the programme's: the artefacts are written on the left, the gates on them run in the middle.

> **ASK**
>
> Which column do you recognise? Where did a gap in a diagram last cost you time?

Next → What do we mean by "makes sense"?

What we mean by semantic correctness

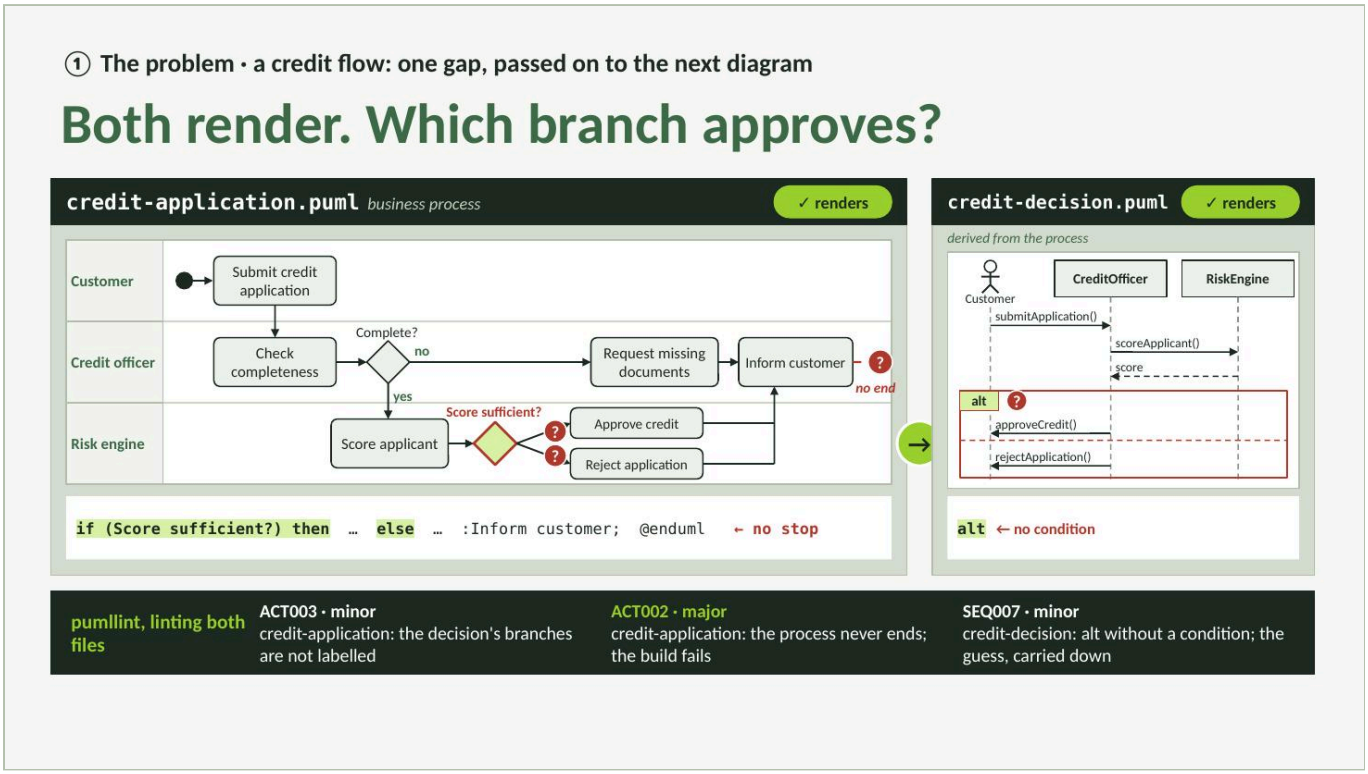


SAY

- Syntax sits underneath and is solved: PlantUML tells you whether the file renders.
- Semantic correctness is six questions a reader would otherwise answer with a guess. Walk them with one example each.
- "Gaps" are Completeness, "ambiguities" are Ambiguity: together 55% of the score, and they decide whether a diagram is ready for hand-off.
- The percentages are the weights pumllint uses to combine the six into one score.

Next → Two examples. First, a credit flow.

Both render. Which branch approves?



SAY

- The first example: part of a credit flow as a process engineer would draw it, and the sequence diagram a designer derived from it. Both render.
- The process asks "Score sufficient?" but never says which branch is yes. The designer had to guess, and the guess shows up downstream as an alt block with no condition.
- pumllint reports both ends of the gap: ACT003 on the process (unlabelled branches) and SEQ007 on the sequence (alt without a condition). ACT002 adds that the process never reaches an end; it is major, so the build fails.
- pumllint does not know your credit policy. It flags that the diagram does not state it.

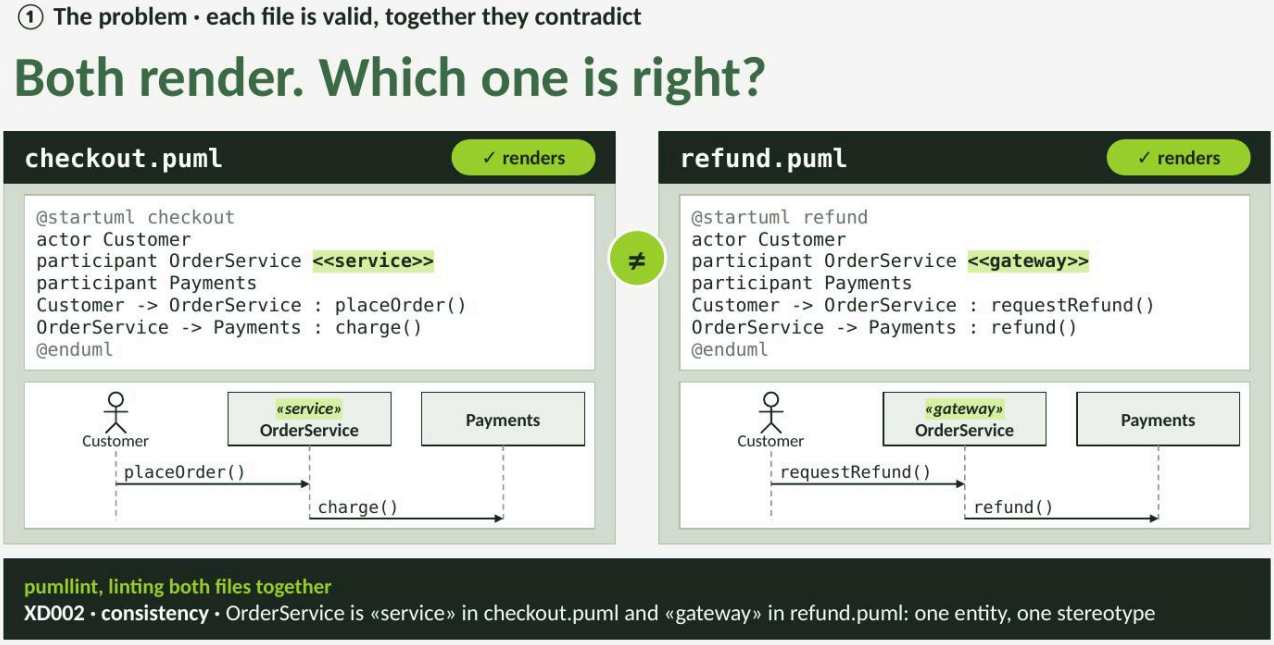
> ### ASK
>
> Before the reveal: which branch approves the credit? Then: who would have caught this today, and when?

TIP

The findings strip appears on the next click: ask first, then click.

Next → A second kind of problem only shows across diagrams.

Both render. Which one is right?



SAY

- The second example: two files, each valid, each renders. The syntax layer is satisfied, file by file.
- Reveal the strip only after the room has answered. Read across both files, pumllint reports XD002, a Consistency finding: one entity, two stereotypes.
- It does not pick a side; it shows both places. A reviewer reading one file at a time never sees it.

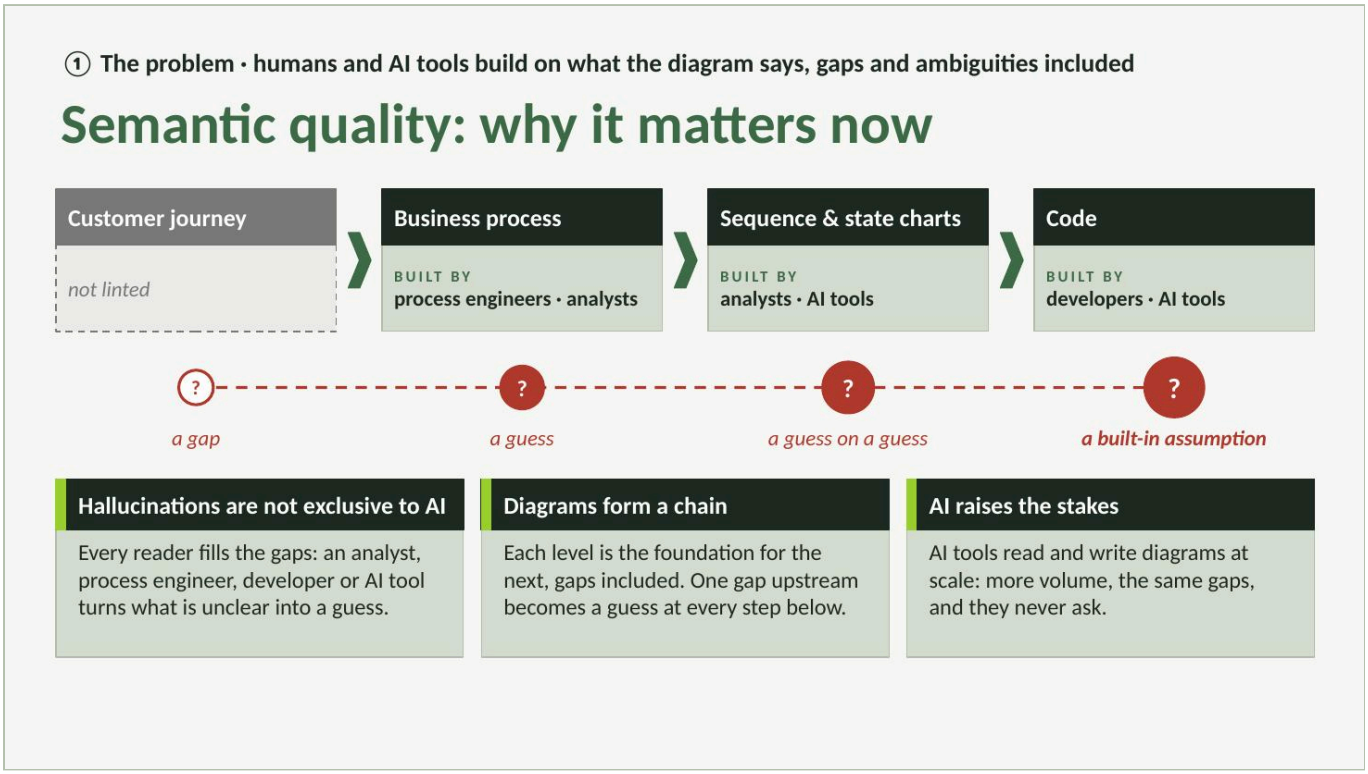
> **ASK**
>
> Before the reveal: which one is right? Is OrderService a service with its own logic, or a gateway that passes calls through? Let two or three people answer.

TIP

The XD002 strip appears on the next click: ask first, then click.

Next → Why this matters: someone builds on every diagram.

Semantic quality: why it matters now

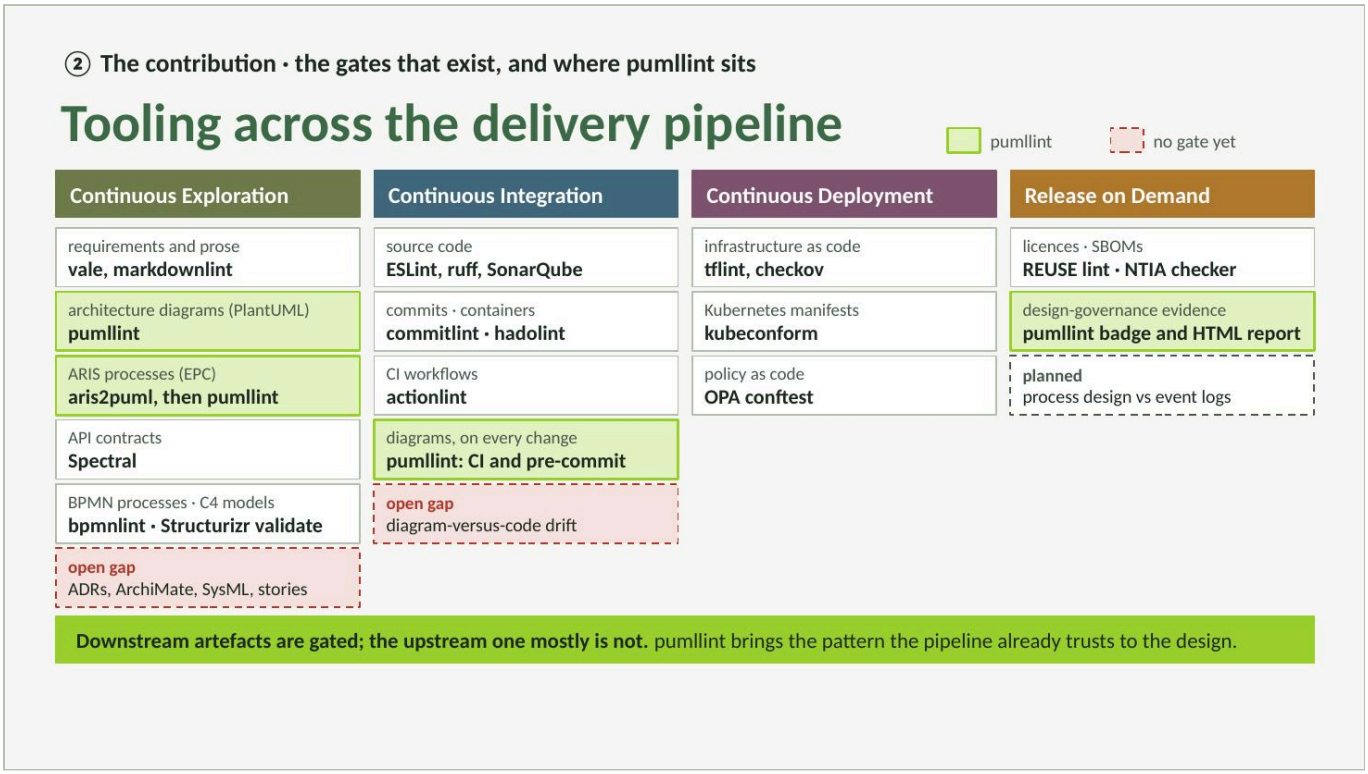


SAY

- The credit flow you just saw is one link of this chain. Walk it from left to right. The customer journey is not linted (there is no PlantUML journey type), so its gaps travel down unchecked.
- At every hand-off someone builds on the step before: process engineers and analysts, then analysts and AI tools, then developers and AI tools.
- Whatever is unclear becomes a guess; the guess becomes the foundation of the next step; in code it is an assumption that compiles and passes review.
- Filling gaps with guesses is not only an AI habit. AI raises the stakes: more volume, the same gaps, and it never asks.

Next → Which tools gate which artefact today?

Tooling across the delivery pipeline



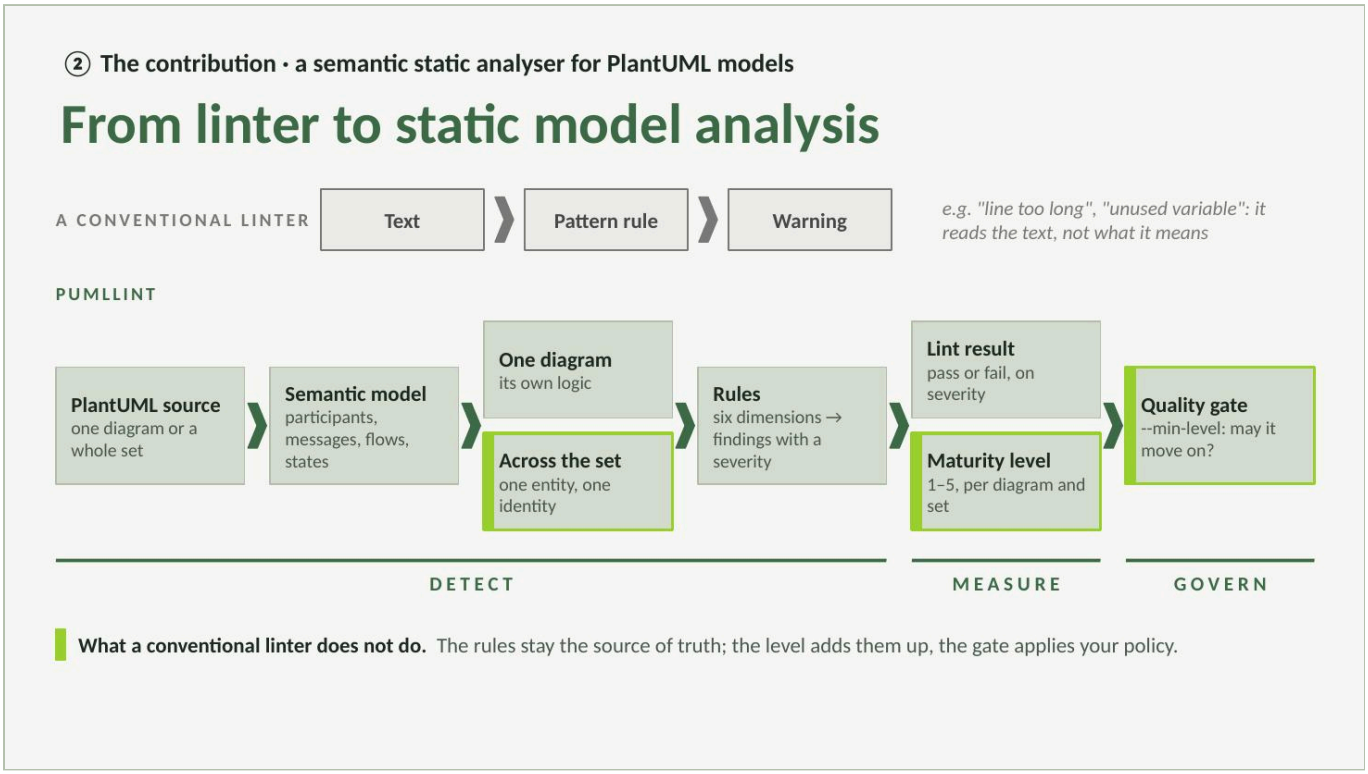
SAY

- The same four columns, now with the deterministic gates that exist for each artefact.
- Everything downstream has one: code, commits, containers, infrastructure, licences. CI runs them without debate.
- Upstream, the gates are patchy: prose and API contracts have them, most models do not. Red boxes are open gaps: ADRs, ArchiMate, SysML, user stories, the glossary.
- The green boxes are pumllint: PlantUML diagrams, ARIS processes converted by aris2puml, the same check on every change, and the evidence for governance.

> **ASK**
>
> Which of these gates already runs in your pipeline, and who looks at its results?

Next → How a semantic linter works.

From linter to static model analysis



SAY

- A conventional linter matches patterns in text: line too long, unused variable.
- pumllint first turns each diagram into a model (who talks to whom, which flows, which states) and reasons over it, inside one diagram and across the whole set. Slide 6's finding only exists across files.
- Detect, findings with a severity, is what any linter does. pumllint goes two steps further: measure (a maturity level per diagram and per set) and govern (a gate the team sets).
- Hence "static model analysis" rather than just a linter.

Next → What do those findings look like?

Code smells, model smells

② The contribution · not always an error, always a reason to guess

Code smells, model smells

Code smell · SonarQube

Something in the code that makes it harder to understand, change or trust; not always a bug.
e.g. too complex · unreachable branch · unclosed resource

≈

Model smell · pumllint

Something in the diagram that makes it harder to interpret, implement or trust: **a reason for the next reader to guess.**

Dimension	Diagram	Model smell	Rule	Severity
Logical correctness	sequence	A participant sends messages but was never declared	SEQ001	**critical**
Completeness	business process	The flow never reaches an end	ACT002	**major**
Ambiguity	state chart	A transition with no label: what triggers it?	STA003	**minor**
Consistency	across the set	One participant is a service here, a database there	XD001	**major**
Traceability	any	No link to the requirement or decision it implements *	GEN007	**minor**
Readability	any	Too many elements on one canvas (over 60)	GEN009	**minor**

** checked once your requirement-tag format is configured*

SAY

- Borrow a term the room knows: a code smell is not always a bug, but it makes code harder to understand or trust.
- A model smell is the same for a diagram: not always an error, always a place where the next reader has to guess.
- One real rule per dimension, across diagram types. For process engineers, also ACT003: a decision whose branches are not labelled.
- Dark severity cells fail the build; the next slide shows what each severity does.

Next → What does each severity do?

Detect, measure, govern

② The contribution · every finding has a severity; every diagram gets a level

Detect, measure, govern

1 DETECT findings

Severity	Meaning	Effect
Blocker	Unusable as a specification	**Fails the build · max Level 2**
Critical	Broken structure	**Fails the build · blocks Level 5**
Major	Breaks a mandatory standard	**Fails the build · blocks Level 5**
Minor	Breaks a recommended convention	Reported · lowers the score
Info	Advisory	Reported · lowers the score slightly

2 MEASURE maturity

Level	Name	Read it as
5	**Method-complete**	Every dimension strong, no majors: preconditions for AI code generation met (codegen profile only)
4	**Precise**	Complete and unambiguous: ready for hand-off
3	**Disciplined**	No blockers: ready for review, not yet for hand-off
2	**Structured**	Coherent enough to work on, although it may still contain blockers
1	**Sketchy**	Don't build on it

3 GOVERN

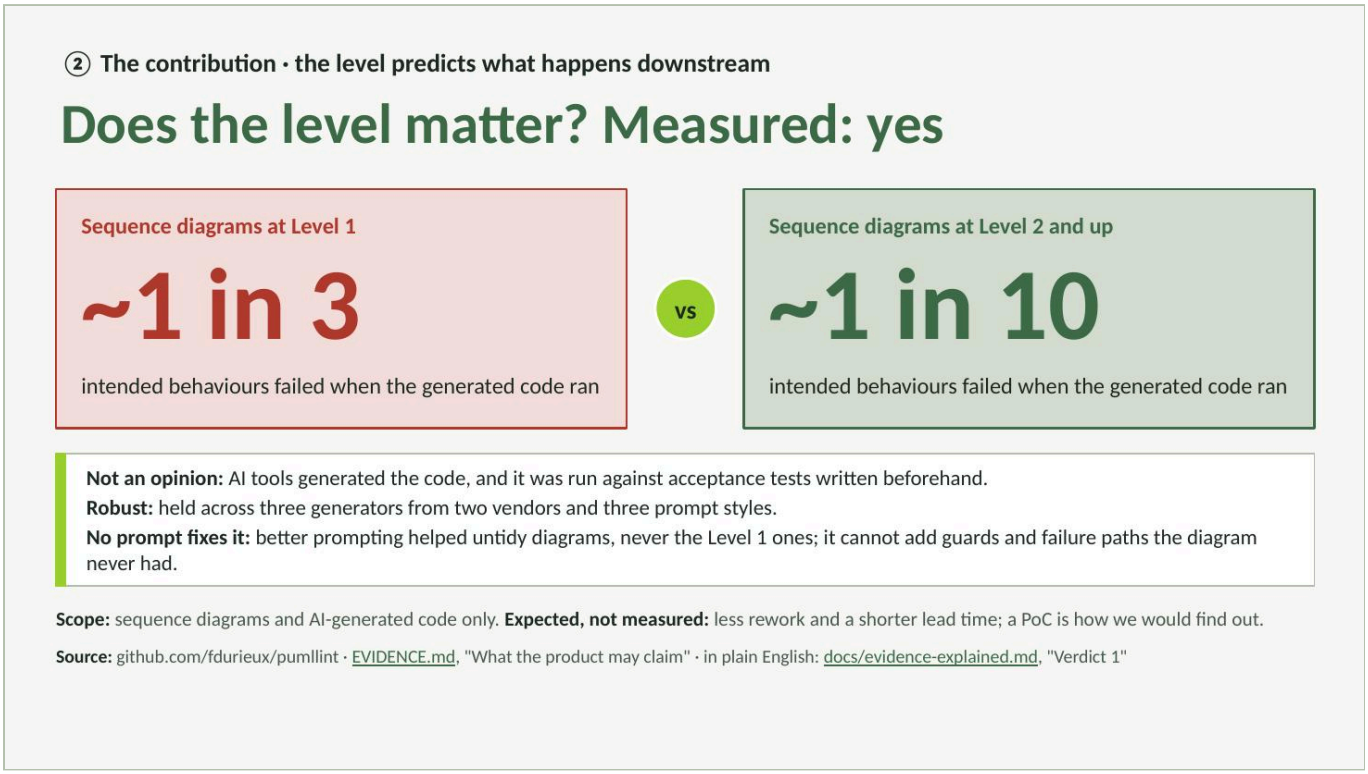
The team sets the bar, e.g. Level 2 everywhere, Level 4 before hand-off: a diagram below it does not move on.
Business processes: gate on findings, not on the level; no ambiguity rule covers activity diagrams yet.

SAY

- Detect: blocker, critical and major fail the build; minor and info are reported and lower the score.
- Measure: the findings add up to a level per diagram. Level 4, Precise, is the hand-off bar: the next step can build on it without guessing.
- Govern: the team picks the bar and the gate enforces it. For business processes, gate on findings: no ambiguity rule covers activity diagrams yet, so their level can look better than it is.
- The demo in a minute shows exactly that: every broken diagram still reaches Level 4, and the gate still fails.

Next → Does the level actually matter?

Does the level matter? Measured: yes



SAY

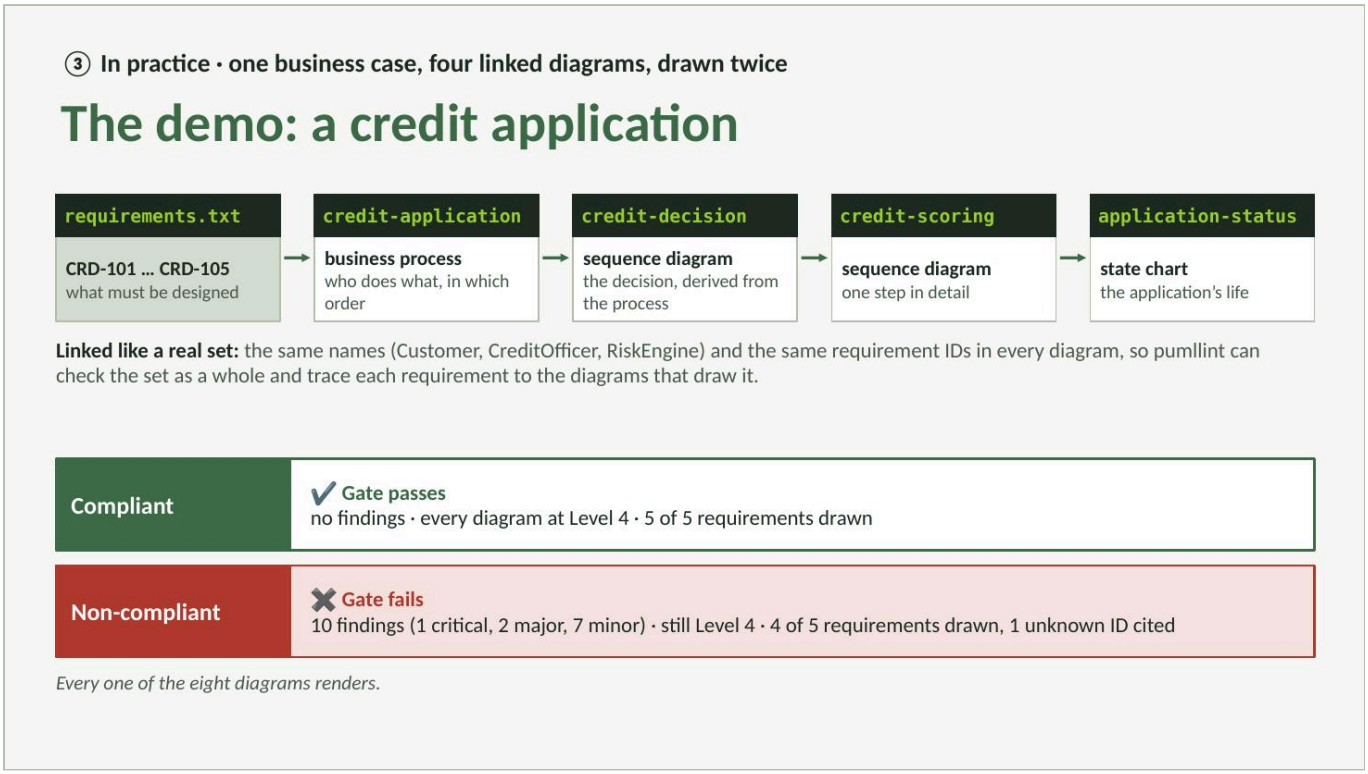
- The one result we measured. AI tools generated code from sequence diagrams, and the code was run against acceptance tests written beforehand.
- From Level 1 diagrams about 1 in 3 intended behaviours failed; from Level 2 and up about 1 in 10. It held across three generators from two vendors and three prompt styles.
- Better prompting lifted moderately untidy diagrams, never the Level 1 ones: a prompt cannot add guards and failure paths the diagram never specified.
- Scope: sequence diagrams and AI-generated code only. Less rework and a shorter lead time are expected, not measured.

TIP

Source, if asked: EVIDENCE.md in the pumllint repository, section "What the product may claim"; the plain-English version is docs/evidence-explained.md, "Verdict 1".

Next → Enough slides: one business case, drawn right and drawn wrong.

The demo: a credit application



SAY

- One business case, four linked diagrams: the requirements, the business process, the decision as a sequence diagram, one step in detail, and the application status as a state chart.
- They share names and requirement IDs, as a real set would, so pumllint can check them together.
- Each diagram exists twice: compliant and with deliberate defects. All eight render.

Next → What is wrong with the second version, and why it matters.

Valid or not, and why

③ In practice · the compliant versions pass; here is what the others get wrong

Valid or not, and why

Diagram	What is wrong in the non-compliant version	Rule · severity	What the next reader has to guess
Business process `credit-application.puml`	The score decision does not name its branches, and the flow never reaches an end.	`ACT003 ×2 · minor` `ACT002 · major`	*Which branch approves? When is an application finished?*
Decision `credit-decision.puml`	An "alt" with no condition; a typo, "Custmer", quietly adds a second customer.	`SEQ007 · minor` `SEQ001 · critical` `XD002 · minor`	*When does each branch apply? Who is the second customer?*
Scoring `credit-scoring.puml`	RiskEngine is a <<gateway>> here and a <<service>> in the decision diagram.	`XD002 · minor`	*Does the risk engine hold the logic, or only pass calls on?*
Status `application-status.puml`	Nothing says what makes an application Approved; nothing leads to Withdrawn; no owner is named.	`STA003 · minor` `STA002 · major` `GEN006 · minor`	*What triggers approval? Can an application be withdrawn? Whom do I ask?*
Requirements	CRD-105 ("the customer is informed") is drawn nowhere; CRD-106 is cited but does not exist.	`trace`	*Is informing the customer designed at all? Typo, or a missing requirement?*

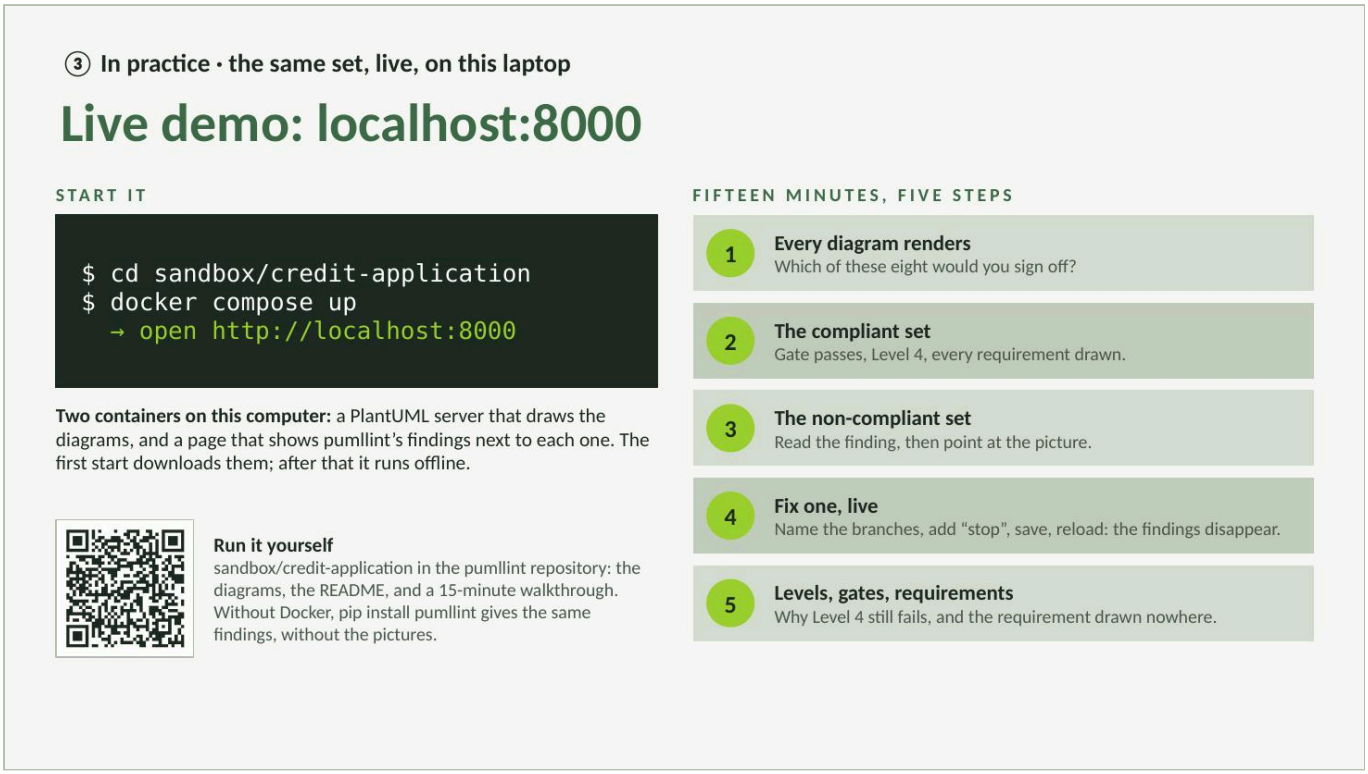
Every non-compliant diagram still scores Level 4. The level says how complete and precise a drawing is overall; the gate looks at how serious each finding is. One critical or major finding stops the hand-off.

SAY

- Read the right-hand column aloud: every defect is a question the next reader has to answer by guessing.
- The rule column is what pumllint reports; the test suite checks it against a live run, so this slide cannot drift from the tool.
- The bottom line answers the first question analysts ask: the level summarises, the gate judges severity.

Next → Now the same set, live.

Live demo: localhost:8000



SAY

- Switch to the browser. Follow WALKTHROUGH.md in the sandbox folder: every diagram renders; the compliant set; the non-compliant set; fix one live; levels, gates and requirements.
- Let the room spot defects in the pictures before you scroll to the findings.
- The live fix is the moment that lands: label the two branches, add "stop", save, reload, and three findings disappear.

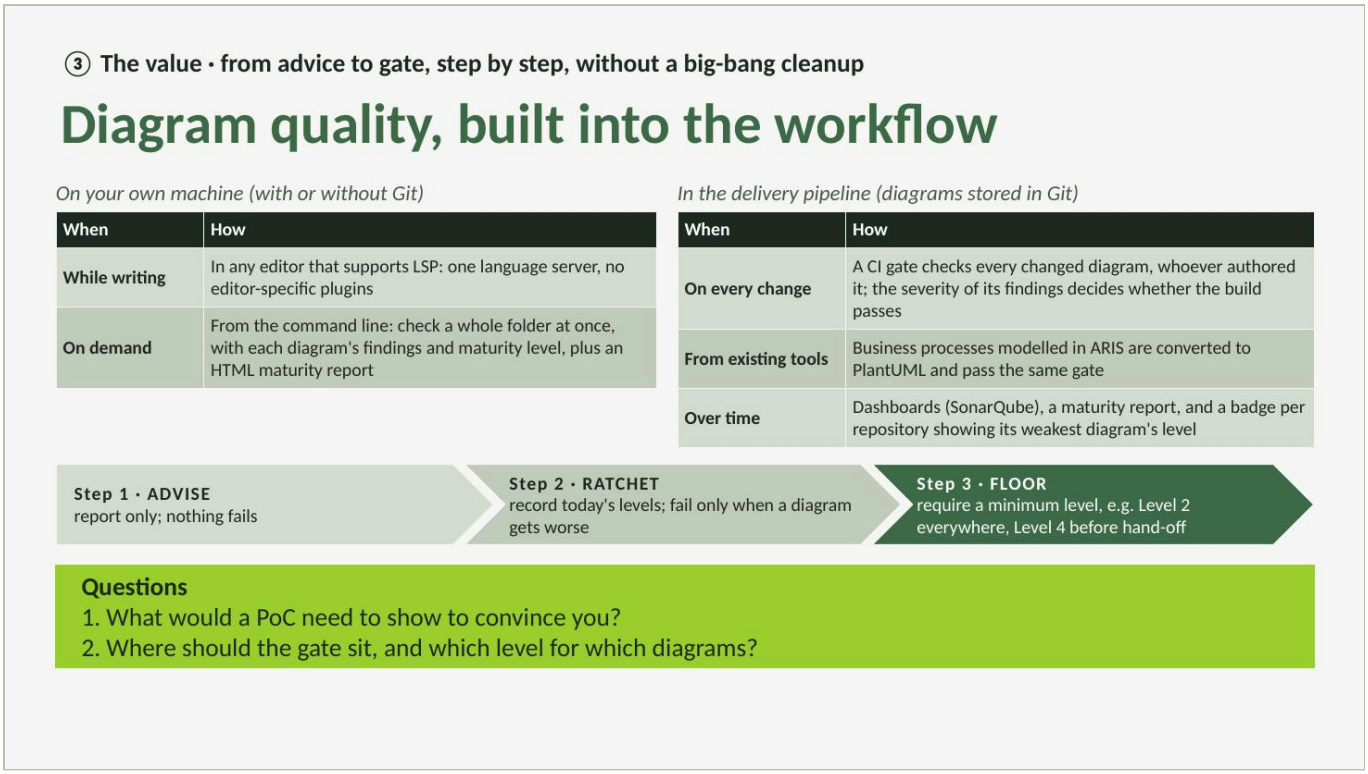
> **ASK**
>
> Who downstream would have found these, and when?

TIP

Start docker compose up before the session. If anything fails, slides 13 and 14 carry the same results.

Next → Where would this run in our work?

Diagram quality, built into the workflow



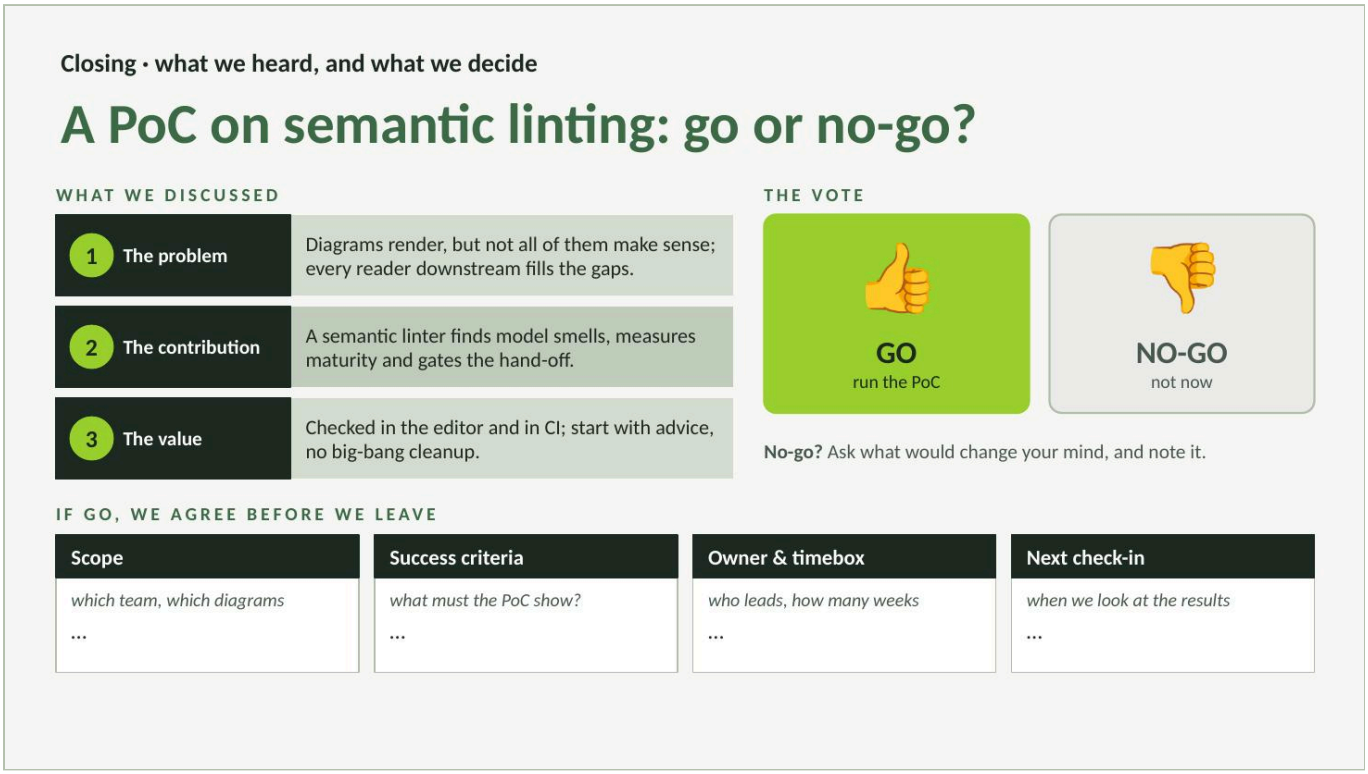
SAY

- Two places, one tool. On your own machine: advice in the editor while you write, a full check from the command line with an HTML report.
- In the pipeline: a CI gate on every change, business processes converted from ARIS through the same gate, and dashboards, a report and a badge over time.
- Roll-out without a big-bang cleanup: advise (nothing fails), ratchet (nothing gets worse), floor (a minimum level).
- A PoC could start at step 1 on one team's diagrams.

> **ASK**
>
> The two closing questions: what would a PoC need to show to convince you? Where should the gate sit, and which level for which diagrams? Collect the answers: they become the success criteria of the PoC on the next slide.

Next → Time to decide.

A PoC on semantic linting: go or no-go?



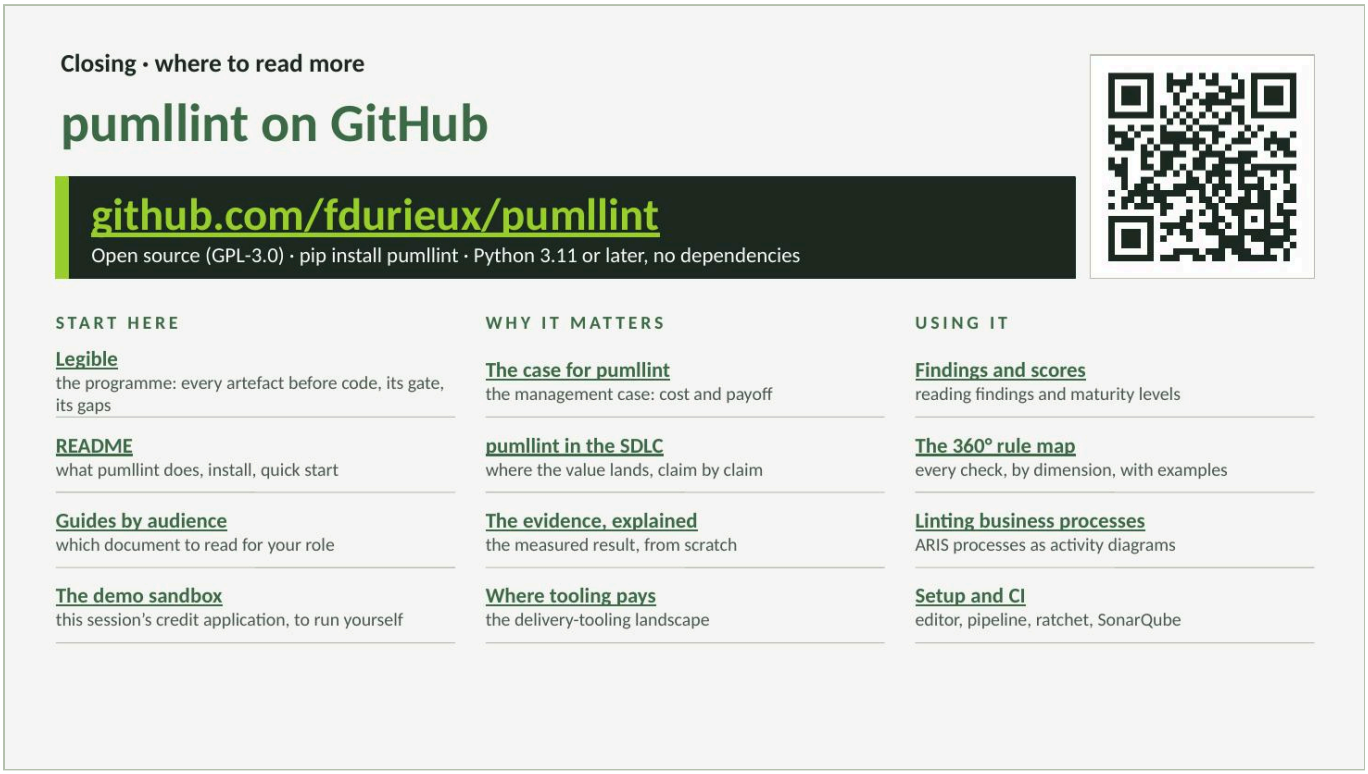
SAY

- One line per part: diagrams render but not all make sense; a semantic linter finds model smells, measures maturity and gates the hand-off; it runs in the editor and in CI, starting with advice.
- Take the vote: thumbs up for go, thumbs down for no-go. Count both.
- If go, fill the four boxes before people leave: scope (one team, its diagrams), success criteria (from the answers on the previous slide), an owner with a timebox, and the date of the next check-in.
- If no-go, ask what would change people's minds and write it down: that is the input for a next attempt.

> ASK
>
> Thumbs up or thumbs down: do we run a PoC? Then, if go: who brings the diagrams, and what must the PoC show?

Next → Where to read more.

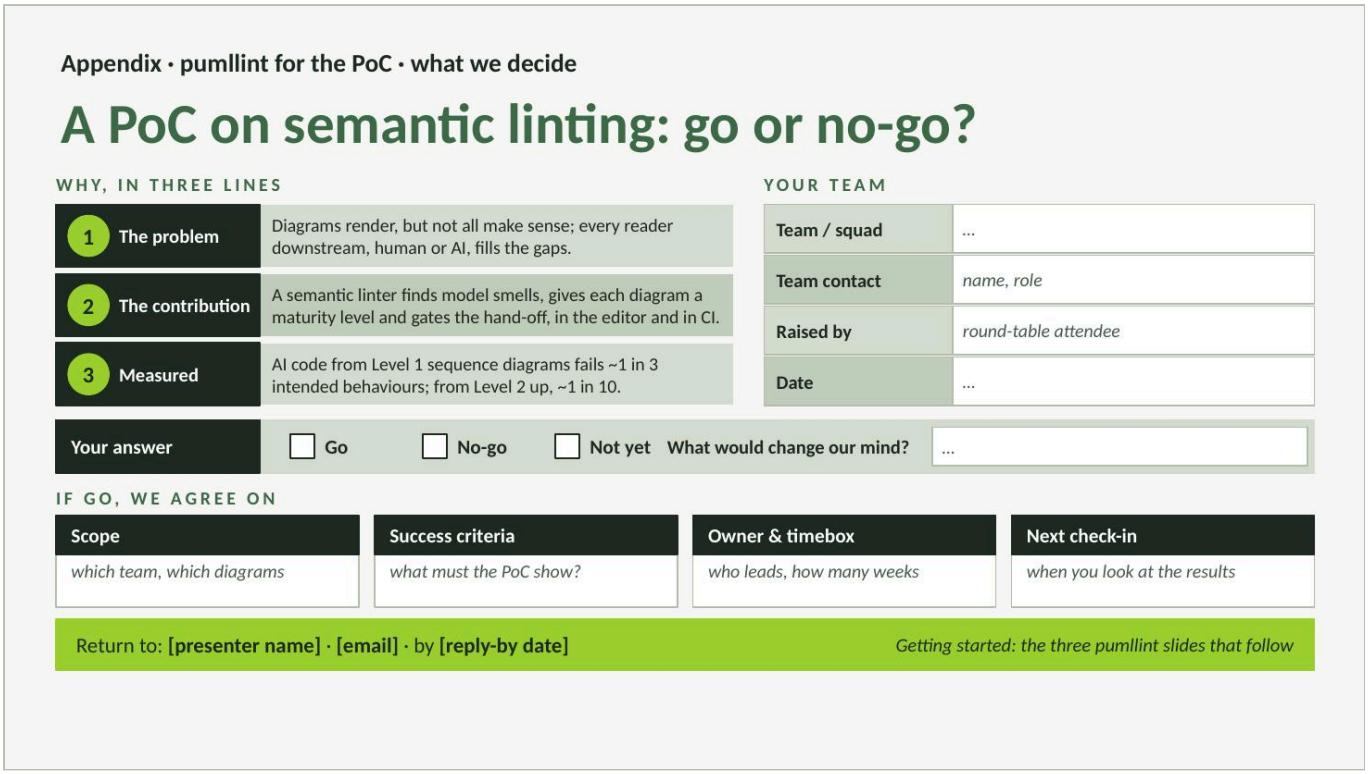
pumllint on GitHub: where to read more



SAY

- Leave this slide up while people leave: the QR code opens the repository; Legible and the demo sandbox are in the first column.

Appendix · A PoC on semantic linting: go or no-go?



Appendix · pumllint for the PoC · what we decide

A PoC on semantic linting: go or no-go?

WHY, IN THREE LINES

1 The problem	Diagrams render, but not all make sense; every reader downstream, human or AI, fills the gaps.
2 The contribution	A semantic linter finds model smells, gives each diagram a maturity level and gates the hand-off, in the editor and in CI.
3 Measured	AI code from Level 1 sequence diagrams fails ~1 in 3 intended behaviours; from Level 2 up, ~1 in 10.

YOUR TEAM

Team / squad	…
Team contact	*name, role*
Raised by	*round-table attendee*
Date	…

Your answer ☐ Go ☐ No-go ☐ Not yet What would change our mind? …

IF GO, WE AGREE ON

Scope	Success criteria	Owner & timebox	Next check-in
which team, which diagrams	*what must the PoC show?*	*who leads, how many weeks*	*when you look at the results*

Return to: **[presenter name]** · **[email]** · by **[reply-by date]** *Getting started: the three pumllint slides that follow*

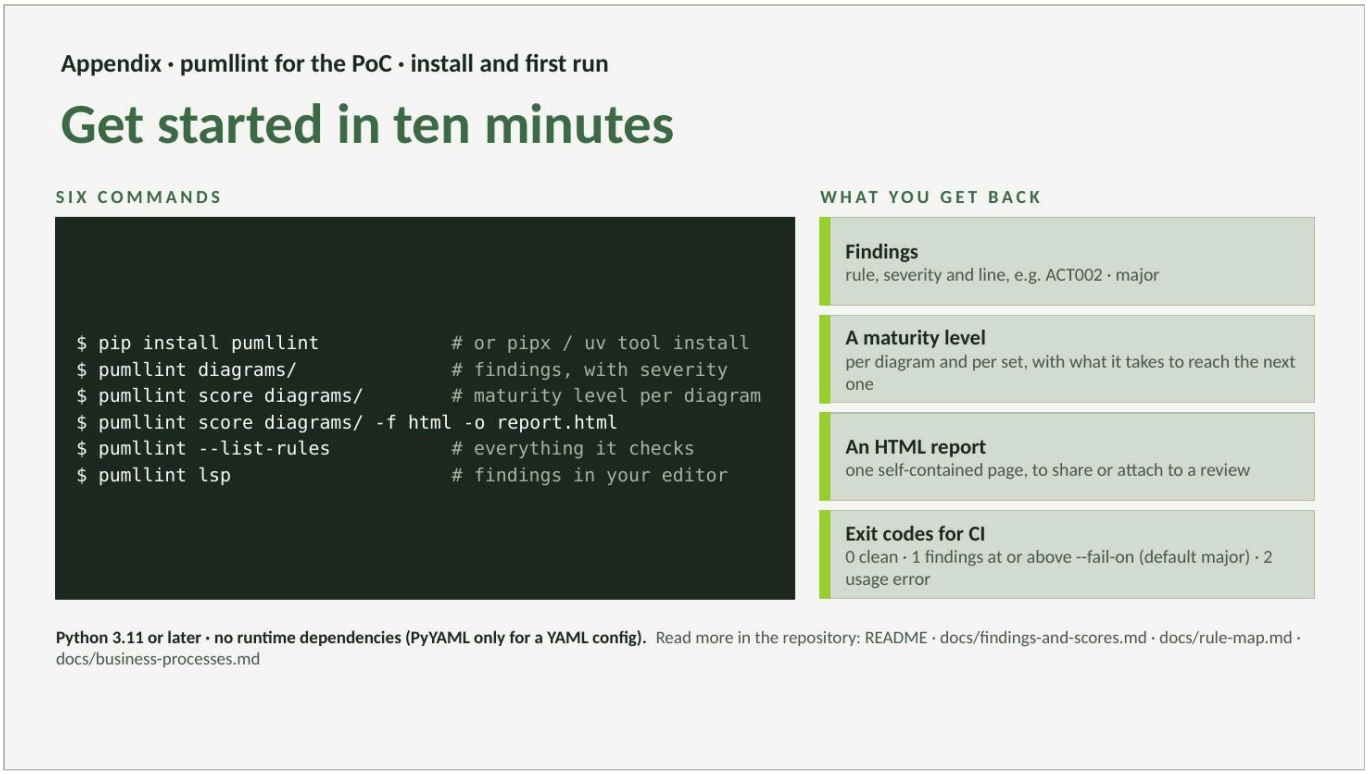
SAY

- Not presented: a one-page form for attendees to take back to their own squad or team lead, together with the three pumllint slides that follow.
- It carries the why in three lines, so a team lead who was not in the room can decide: team, contact, the answer (go, no-go, not yet) and, if go, the same four agreements as on the closing slide.
- Answers go back to [presenter name] by [reply-by date]; the address is in the lime strip.

TIP

Send slides 19 to 22 as one PDF (team-handout.pdf), or print them.

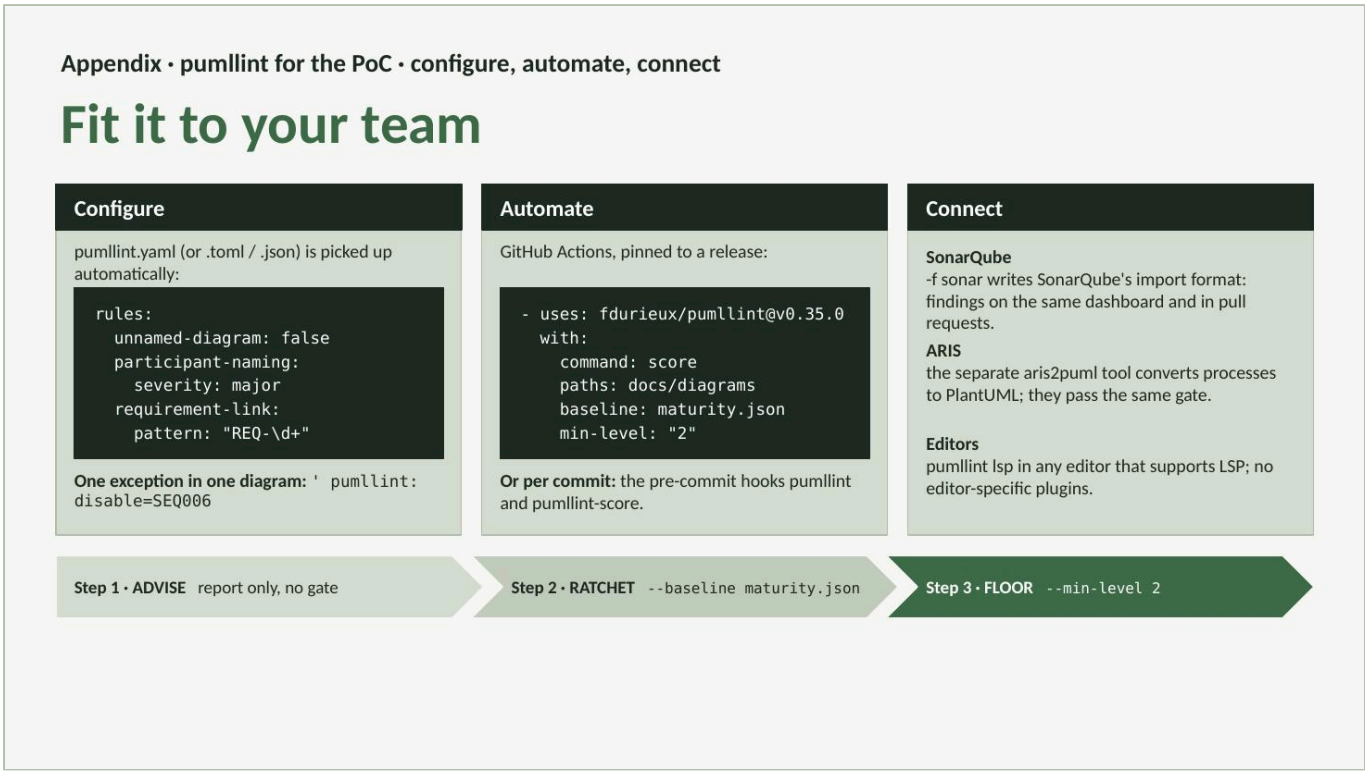
Appendix · Get started in ten minutes



SAY

- Not presented: hand-out for the PoC team. Six commands from the README quick start cover a first run.

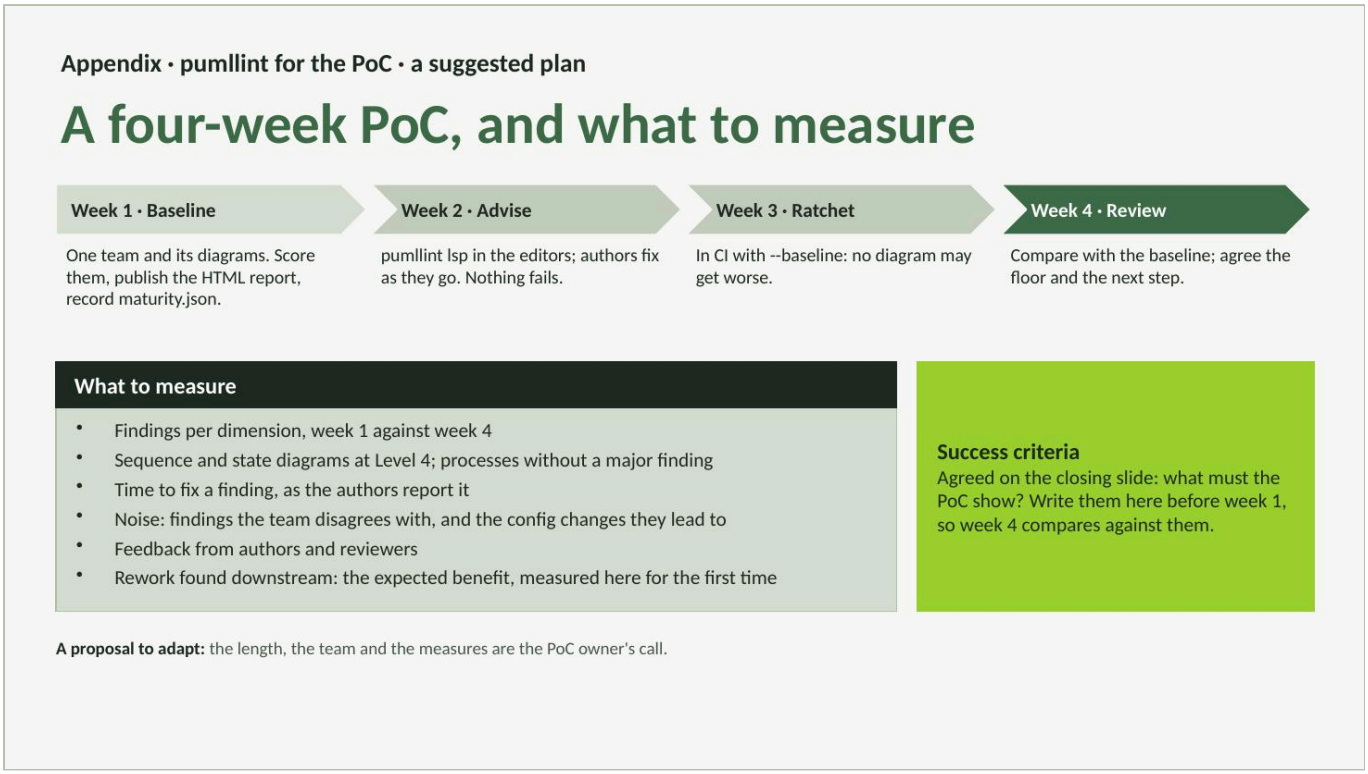
Appendix · Fit it to your team



SAY

- Not presented: hand-out for the PoC team. One config file, a GitHub Action or pre-commit hooks, and the roll-out mapped to three commands.

Appendix · A four-week PoC, and what to measure



SAY

- Not presented: hand-out for the PoC team. A suggested plan; the measures give the expected benefits a first measurement.